从0开发大模型的17种Agent架构演进详细拆解

腾讯技术工程 2026年5月18日 02:39 广东 75人



作者：linkxzhou，公众号《周末程序猿》

在 github 看到一个项目 `all-agentic-architectures`，实现的是关于 17 种 Agent 模式大全，我把 `all-agentic-architectures` 这个项目完整过了一遍。它最有价值的地方，不是列了 17 中 Agent 代码，而是把 17 次系统升级拆解：上一个架构为什么不够、下一个架构到底多了什么控制能力、复杂度又是从哪里开始失控的。这些信息真正决定一个 agent 系统能不能落地的，通常不是模型回答是不是够好，而是：

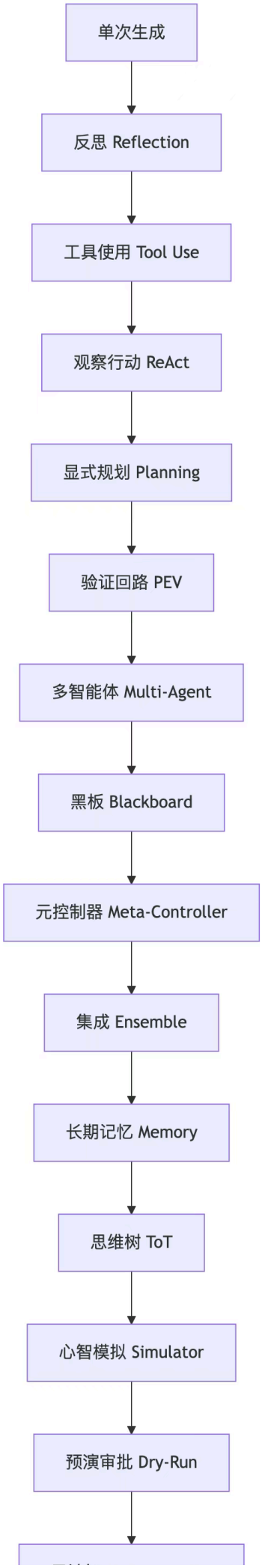
- 状态有没有被正确建模
- 控制流有没有被显式表达
- 错误能不能被局部截断
- 副作用能不能被关进闸门
- 系统知不知道自己什么时候该停

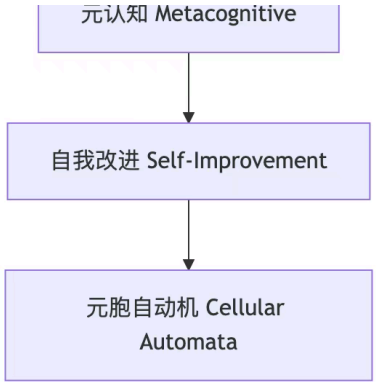
不过 `all-agentic-architectures` 是用 langchain 和 langgraph 实现的，但是 `agno` 框架更加简洁，于是 **用 agno 把这 17 种控制流从头写一遍。** 为了说明一件事：

Agent architecture 的本质不是 prompt engineering，也不是某个框架的 DSL，而是控制流设计。 它应该能在任何体面的 agent 框架里复现，以下是 Agent 演化路径：

- 从**单次生成**到**反思闭环**
- 从**反思闭环**到**工具交互**
- 从**工具交互**到**观察-行动循环**
- 从**局部决策**到**显式规划**
- 从**无验证执行**到**验证驱动重规划**
- 从**单 agent**到**多 agent 编排**
- 从**短期上下文**到**长期记忆系统**
- 从**线性推理**到**搜索、模拟与涌现计算**
- 从**能做事**到**可信任**

总演化图：


单次生成
反思 Reflection
工具使用 Tool Use
观察行动 ReAct
显式规划 Planning
验证回路 PEV
多智能体 Multi-Agent
黑板 Blackboard
元控制器 Meta-Controller
集成 Ensemble
长期记忆 Memory
思维树 ToT
心智模拟 Simulator
预演审批 Dry-Run



1. 统一分析框架

后面每一种架构，我都会用同一套问题来拆。

六个固定问题

1. **它要解决什么问题？** 上一代架构哪里不够。
2. **它的 State 是什么？** 新增了哪些字段，为什么必须存在。
3. **它的拓扑是什么？** 线性链、循环、分叉汇聚、共享黑板、树搜索还是网格涌现。
4. **它的 Router 怎么工作？** 固定边、条件边、动态调度、验证回路、人工审批。
5. **它的失败模式是什么？** 架构最容易在哪个环节坏掉。
6. **什么时候该升级到下一种？** 当前模式的能力边界在哪里。

2. 用 agno 看 Agent

这个项目几乎所有架构都能被 agno 的几个抽象表达清楚：

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.workflow.v2 import Workflow, Step, Router, Loop

# 一个最简 Agent = 一次状态变换
agent = Agent(
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[...],
    instructions="...",
    response_model=SomePydanticModel,  # 结构化输出
)

# 一个 Workflow = 显式控制流
wf = Workflow(
    name="my_flow",
    steps=[
        Step(name="plan", agent=planner_agent),
        Loop(
            name="execute_and_verify",
```

```
            steps=[executor_step, verifier_step],
            end_condition=lambda outputs: outputs[-1].content.is_done,
        ),
        Step(name="synthesize", agent=synthesizer_agent),
    ],
)
wf.run(message="...")
```

这段代码背后已经包含了一个 agent 系统的最小数学结构：

- `response_model` 定义状态空间
- `Agent / tools / Step` 函数定义状态变换
- `steps` 列表定义确定性转移
- `Router / Condition` 定义条件转移
- `Loop.end_condition` 定义终止条件
- `wf.run()` 定义可执行系统

后面我几乎不把问题描述成"这个架构更聪明"，而描述成：它新增了什么状态字段、什么 agent 或工具、什么路由逻辑、什么验证机制。

3. 总览：逐步添加控制能力

阶段	新增能力	一句话解释	代表架构
单次生成优化	critique pass	让模型先出一版，再自己挑毛病改掉，把"生成"拆成 generator + critic + refiner 三步。	Reflection
与世界交互	tool interface	把外部 API / 函数挂成结构化工具接口，打破参数知识边界和上下文封闭性。	Tool Use
基于观察持续行动	observation loop	Thought → Action → Observation 滚动循环，上一步的观察决定下一步的动作。	ReAct
先生成控制流再执行	explicit planning	先让模型产出一份可检视的步骤清单（plan），再按清单逐步执行，把控制流本身做成可审计对象。	Planning
把验证接入主回路	verification loop	每一步执行完都强制过一次 verifier，失败就回到重规划，而不是把验证当成事后检查。	PEV
把认知任务拆成角色	role decomposition	不再让一个 agent 什么都干，把研究员 写作 审阅等角色拆开，用流水线或图把它们串起来。	Multi-Agent
把中间状态显式共享	shared workspace	把中间产物写到一块共享"黑板"上，一个 controller 根据黑板当前状态动态决定下一个该谁上场。	Blackboard
把入口做成路由系统	entry routing	在请求入口先分类一次，把任务路由到最合适的专家子 agent，避免一个 agent 背所有能力。	Meta-Controller

阶段	新增能力	一句话解释	代表架构
用冗余换可靠性	parallel redundancy	同一个问题同时交给多个独立 agent 处理，再由 aggregator 融合/投票，用冗余换稳定性和去偏。	Ensemble
把历史状态纳入系统	long-term memory	把对话片段放进向量库（episodic），把结构化事实放进图/KV（semantic），让系统记得住、查得到。	Episodic + Semantic / Graph
把推理变成搜索	search tree	不再是一条思路走到底，而是展开多条思路形成树，边展开边打分、剪枝，把"推理"变成"搜索"。	ToT
把行动前评估做成模拟	counterfactual execution	真正动手之前，先在内部世界模型里预演一遍，评估风险和收益，再决定是否真执行。	Mental Loop
把副作用关进闸门	side-effect gating	任何有副作用的动作必须先 dry-run + 审核（人或自动检查）通过后，才允许落到真实环境。	Dry-Run
把自我边界建模	self-boundary reasoning	系统维护一份自我模型，知道自己"擅长什么、不擅长什么"，据此选择亲自做、调工具，还是交给人。	Metacognitive
把质量改进做成循环	iterative refinement loop	editor agent 对输出打分并给出修改意见，writer 按意见改稿，高分样本沉淀下来用于持续改进。	Self-Improve
去中心化计算	emergence	没有中心 LLM 在推理，每个格子/单元只跑简单局部规则，全局行为（寻路、扩散等）从局部交互中涌现出来。	Cellular Automata

所谓"agent 架构演化"，不是追求 AGI，而是解决一个老问题：**怎样让系统在更复杂的环境里，依然保持可控、可解释、可恢复。**

下面正式开始。为了篇幅，代码里我统一省略 `from agno... import` 的重复部分。

4. Reflection：最小质量闭环

`Reflection` 的价值不在于"模型会反思"，而在于它第一次把生成过程拆成了两个不同职能的 pass：生成和评估，它是最小质量闭环，但还不是完整控制闭环。

它要解决什么问题？

单次 LLM 生成质量不稳定，Reflection 就是最小修复：先生成 → 再评估 → 再根据评估修改。这不是"增加智能"，而是**把单步生成改成三阶段控制流**。

它的 State 是什么？

核心三个字段是 `draft / critique / refined_code`，在 agno 里直接用 Pydantic 把它们定义成结构化输出：

```python
from pydantic import BaseModel, Field
from typing import List

class DraftCode(BaseModel):
    code: str = Field(description="Python code to solve the user's request.")
    explanation: str = Field(description="A brief explanation of how the code works.")

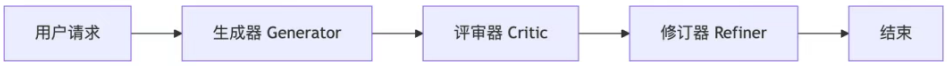
class Critique(BaseModel):
    has_errors: bool
    is_efficient: bool
    suggested_improvements: List[str]
    critique_summary: str

class RefinedCode(BaseModel):
    refined_code: str
    refinement_summary: str
```

系统第一次把"中间思考结果"显式写进 state，而不是埋在上下文里，`response_model=...` 让这件事几乎零成本。

它的拓扑是什么？

纯线性：



用 agno 写出来：

```python
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.workflow.v2 import Workflow, Step

model = OpenAIChat(id="gpt-5-mini")

generator = Agent(name="generator", model=model, response_model=DraftCode,
    instructions="You are an expert Python programmer. Write code and a brief explanation

critic = Agent(name="critic", model=model, response_model=Critique,
    instructions="You are a senior code reviewer. Analyze for bugs, inefficiencies and PE

refiner = Agent(name="refiner", model=model, response_model=RefinedCode,
    instructions="Rewrite the code, incorporating every suggestion from the critique.")

def generator_step(step_input):
    draft = generator.run(step_input.message).content
```

```
    step_input.workflow_session_state["draft"] = draft
    return draft

def critic_step(step_input):
    draft: DraftCode = step_input.workflow_session_state["draft"]
    return critic.run(f"Review this code:\n```python\n{draft.code}\n```").content

def refiner_step(step_input):
    draft: DraftCode = step_input.workflow_session_state["draft"]
    critique: Critique = step_input.previous_step_output.content
    prompt = (f"Original code:\n```python\n{draft.code}\n```\n"
              f"Critique: {critique.model_dump_json(indent=2)}\nProduce the refined code.
    return refiner.run(prompt).content

reflection_wf = Workflow(
    name="reflection",
    session_state={"draft": None},
    steps=[
        Step(name="generator_step", executor=generator_step),
        Step(name="critic_step", executor=critic_step),
        Step(name="refiner_step", executor=refiner_step),
    ],
)

reflection_wf.run(message="Write a Python function to find the nth Fibonacci number.")
```

`Workflow.steps` 列表里的顺序就是确定性边：上一步的输出自动作为下一步的 `previous_step_output`，每一步的 state 流动由 agno 托管——`previous_step_output` 是一个 `StepOutput`，它的 `.content` 才是上一步 `response_model` 的实例；跨非相邻步骤的数据用 `workflow_session_state` 显式传递。

Router 怎么工作？

没有 router。 这是 Reflection 很关键的技术边界：没有条件分支，没有失败恢复，系统默认三步走完直接结束。

它为什么比单次生成强？

因为它把一个模糊任务拆成了三个更明确的子任务：先尽量生成 → 再专门找问题 → 最后只解决问题。这里的核心经验判断是：**LLM 作为 critic 往往比作为 generator 更稳定。**

它的失败模式

最大的问题是：**它不能验证 refiner 是否真的修好了 critic 提到的问题。** 它有 critique，但没有闭环。

什么时候升级？

当你需要系统根据中间结果继续行动，就必须进入下一代：要么让它接触世界（Tool Use），要么形成持续观察-行动回路（ReAct）。

> 📎 参考实现：**01_reflection.ipynb**

5. Tool Use：文本世界到结构化世界的跨越

它要解决什么问题？

Reflection 解决"质量"，但没解决"知识边界"，一个不带工具的 LLM 再会反思也被困在参数里，Tool Use 要**让系统突破上下文与知识截止日期的封闭性**。

它的 State 是什么？

Tool Use 的 state 本质上是一条"事件日志"：用户输入 → 模型输出 → 工具调用 → 工具返回 → 下一轮推理。 在 agno 里，这条日志是 `Agent` 内部自动维护的——一次 `agent.run()` 内部就完整地跑完这整个链路。

state 从"你需要自己维护的数据结构"变成了"框架托管的会话上下文"，这也是 Tool Use 和 Reflection 在工程侧最显眼的差别。

它的关键代码

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools

def get_stock_price(symbol: str) -> str:
    """Return the latest stock price for a given symbol."""
    return f"The current price of {symbol.upper()} is $172.35."

tool_agent = Agent(
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[get_stock_price, DuckDuckGoTools()],
    instructions="Use tools to answer questions that need real-time data.",
    show_tool_calls=True,
)

tool_agent.run("What is Apple's current stock price?")
```

底层等价于：

```
# 概念上 agno 内部循环
while True:
    response = llm_with_tools.invoke(messages)
    messages.append(response)
```

```
    if not response.tool_calls:
        break
    for call in response.tool_calls:
        observation = tool_registry[call.name](**call.args)
        messages.append(ToolMessage(observation, tool_call_id=call.id))
```

这个 while 循环是 agno `Agent` 帮你内置的。

它真正新增了什么能力？

不是"会调用函数"，而是：**文本控制流可以跨越到结构化世界，再返回文本世界。** 这个跨越点是整个 agent engineering 的第一道硬边界。

失败模式

失败通常不来自 LLM 本身，而来自"边界层"：工具名幻觉、参数类型错误、工具返回格式不对、工具结果被模型错误综合。Tool Use 的关键难点不是 prompt，而是**序列化与反序列化**。

它为什么还不够？

因为 Tool Use 没有显式规定"一定要循环"，在最原始的实现里，模型可能只调一次工具就收尾，你需要真正的观察-行动闭环——这就是 ReAct。

> 📎 参考实现：**02_tool_use.ipynb**

6. ReAct：Agent 真正成形的地方

它要解决什么问题？

Tool Use 的控制流还太浅，ReAct 要**让工具结果不只是被消费一次，而是进入下一轮决策**。从这里开始，agent 不再只是"会用工具"，而是"会根据新观察更新计划"。

State 还是消息序列，为什么语义却变了？

在 ReAct 里 agent 的消息序列不再只是对话历史，而是**行动轨迹（trace）**：当前问题 → 上一步推理 → 工具调用 → 工具观测 → 新一轮判断。 你可以把它看成隐式的工作记忆带，在 agno 里，这条 trace 是 `Agent` 内部自动维护的——一次 `agent.run()` 内部就包含用户输入、模型输出、工具调用、工具返回、二次推理，你不需要手写任何状态容器。

它的关键代码

agno 内置的 `reasoning=True` 会把 ReAct 的"Thought Action Observation"显式写进 trace：

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools
```

```python
from agno.tools.yfinance import YFinanceTools

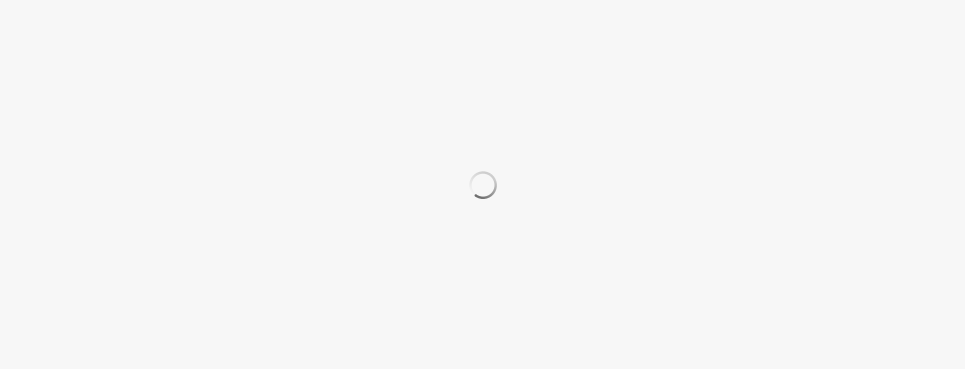
react_agent = Agent(
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[DuckDuckGoTools(), YFinanceTools(stock_price=True, company_news=True)],
    instructions=[
        "You are a research assistant.",
        "Think step by step. For each step decide whether to use a tool or answer.",
        "After each tool observation, re-evaluate what you still need before answering.",
    ],
    reasoning=True,
    markdown=True,
    show_tool_calls=True,
)

react_agent.print_response(
    "Based on the latest news, should I be worried about AAPL next quarter?",
    stream=True,
)
```

关键不是某一行代码，而是 agno 内部那条"回边"：**只要 model 回复里还有 tool_calls，就回到 model 再跑一次。** 这条 `tool → model` 的循环边，是整个 agent architecture 里最重要的回边之一——它把"单次调用"变成了"持续交互"。

拓扑本质

最小闭环：



系统终于从"线性流程"进入"持续交互系统"。

它为什么是 80% 任务的起点？

因为大多数任务并不需要复杂规划，它们只需要：先看一眼 → 做个动作 → 根据反馈再决定下一步，已足够覆盖多轮搜索、多跳问答、网页研究、工具驱动的数据收集。

失败模式

局部贪心。 它每次只基于当前 observation 决策，容易走弯路、重复搜索、陷入局部最优、无法提前安排多步任务。

什么时候升级?

当任务需要显式的步骤顺序控制，就该上 Planning。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号

> 参考实现：**03_ReAct.ipynb**

7. Planning：把控制流本身变成模型输出

它要解决什么问题?

ReAct 本质是在线贪心策略，对需要顺序约束、步骤依赖和过程可追踪的任务，它开始不够用。Planning 要**把原本隐式存在模型脑子里的执行顺序，显式拿出来，写进 state**。

系统第一次把"控制流"对象化了。

State 新增了什么?

```
class Plan(BaseModel):
    steps: List[str] = Field(description="Ordered list of tool/sub-questions.")
```

workflow 的 `session_state` 里还要带上中间结果：

```
session_state = {"plan": [], "intermediate": []}
```

在 ReAct 里下一步做什么是临时决定的，在 Planning 里下一步做什么**先被生成出来**，然后才执行。

关键代码

agno 的 `Workflow` 提供 `Loop` step，天然表达"循环执行直到 plan 空"：

```
from agno.workflow.v2 import Workflow, Step, Loop
from agno.tools.duckduckgo import DuckDuckGoTools

planner = Agent(name="planner", model=OpenAIChat(id="gpt-5-mini"), response_model=Plan,
    instructions="Decompose the user request into a list of atomic tool-queryable steps."
```

```
executor = Agent(name="executor", model=OpenAIChat(id="gpt-5-mini"),
    tools=[DuckDuckGoTools()],
    instructions="Answer exactly one sub-question using tools.")

synthesizer = Agent(name="synthesizer", model=OpenAIChat(id="gpt-5-mini"),
    instructions="Combine intermediate findings into a final answer.")

def plan_step(step_input):
    plan: Plan = planner.run(step_input.message).content
    step_input.workflow_session_state["plan"] = list(plan.steps)
    step_input.workflow_session_state["intermediate"] = []
    return plan

def execute_step(step_input):
    state = step_input.workflow_session_state
    next_q = state["plan"].pop(0)
    obs = executor.run(next_q).content
    state["intermediate"].append(f"Q: {next_q}\nA: {obs}")
    return obs

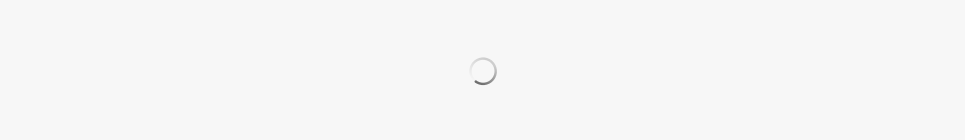
def synth_step(step_input):
    state = step_input.workflow_session_state
    notes = "\n\n".join(state["intermediate"])
    return synthesizer.run(
        f"Question: {step_input.message}\nNotes:\n{notes}\nFinal answer:"
    ).content

def plan_is_empty(_outputs) -> bool:
    """Loop 终止条件：agno 的 end_condition 只接收一个参数（StepOutput 列表）。"""
    return len(planning_wf.session_state["plan"]) == 0

planning_wf = Workflow(
    name="planning",
    session_state={"plan": [], "intermediate": []},
    steps=[
        Step(name="plan", executor=plan_step),
        Loop(
            name="execute_all",
            steps=[Step(name="execute_one", executor=execute_step)],
            end_condition=plan_is_empty,
        ),
        Step(name="synthesize", executor=synth_step),
    ],
)

planning_wf.run(message="Compare the latest revenue of AAPL and MSFT and explain the gap.
```

路由逻辑从"LLM 在 prompt 里决定下一步"变成了"一个数据结构是否还有剩余项"——这才是 Planning 真正的结构变化。



新增的核心能力

Planning 新增的不是"更聪明的思考"，而是：**把未来控制流提前 materialize 成一个数据结构。** 你现在可以可视化计划、检查计划、修改计划、追踪执行进度。

失败模式

它过于乐观，一旦 plan 错了，后面每一步都可能是错的。**可预测性增强了，适应性下降了。**

什么时候升级？

当你不再相信工具会稳定成功，就必须上 PEV。

> 📎 参考实现：**04_planning.ipynb**

8. PEV：把"验证"提升为控制流的一等公民

它要解决什么问题？

Planning 默认世界是稳定的，但真实世界不是：API 会失败、搜索会返回噪音、数据会缺失、工具会超时。

PEV（Plan → Execute → Verify）解决的是：**不要把执行结果默认为真，而要显式验证。** 系统现在不再把执行视为"完成了一步"，而是把执行视为"生成了一个待验证结果"。

State 新增了什么？

```
class VerificationResult(BaseModel):
    is_successful: bool = Field(description="True if the tool execution was successful an
    reasoning: str = Field(description="Reasoning for the verification decision.")
```

verifier 不是随便说一句"好像不对"，而是输出一个结构化 verdict。`verification_result` 字段一出现，图结构就不再是 `plan -> execute -> next step`，而是 `plan -> execute -> verify -> (continue | replan | finish)`。

关键代码

为了演示为什么需要验证，故意放了一个会失败的工具：

```python
def flaky_web_search(query: str) -> str:
    """Search the web. This tool is intentionally unreliable."""
    if"employee count"in query.lower():
        return"Error: Could not retrieve data. The API endpoint is currently unavailable.
    return f"Mock search result for: {query}"

verifier = Agent(
    name="verifier",
    model=OpenAIChat(id="gpt-5-mini"),
    response_model=VerificationResult,
    instructions=(
        "Given a sub-question and the raw tool observation, decide if the "
        "observation actually answers the sub-question. Treat 'Error', "
        "'unavailable', empty strings and obviously irrelevant text as failures."
    ),
)

pev_executor = Agent(
    name="pev_executor",
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[flaky_web_search],
    instructions="Answer exactly one sub-question using tools.",
)

def pev_execute(step_input):
    state = step_input.workflow_session_state
    next_q = state["plan"][0]
    obs = pev_executor.run(next_q).content
    state["last_obs"] = obs
    state["last_q"] = next_q
    return obs

def pev_verify(step_input):
    state = step_input.workflow_session_state
    verdict: VerificationResult = verifier.run(
        f"Sub-question: {state['last_q']}\nObservation:\n{state['last_obs']}"
    ).content
    if verdict.is_successful:
        state["plan"].pop(0)
        state["intermediate"].append(f"Q: {state['last_q']}\nA: {state['last_obs']}")
        state["retries"] = 0
    else:
        state["retries"] = state.get("retries", 0) + 1
    state["last_verdict"] = verdict
    return verdict

from agno.workflow.v2 import Router
```

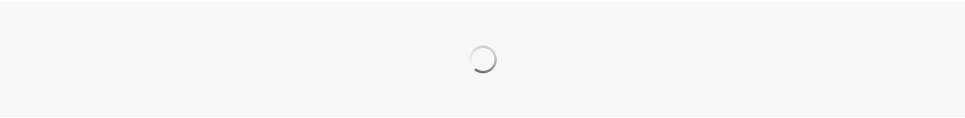
```python
# 先把所有可能用到的 Step 物化一次——Router selector 只能返回 workflow 已知的 Step
replan_step = Step(name="replan", executor=plan_step)
noop_step = Step(name="noop", executor=lambda si: "continue")

def pev_router(step_input):
    """根据本轮 verdict 决定下一轮是否需要重规划；不需要就放 noop，交给 Loop 判断是否再来一轮。"""
    state = step_input.workflow_session_state
    if not state["plan"]:
        return [noop_step]  # 外层 Loop 的 end_condition 会看到 plan 空，直接终止
    if not state["last_verdict"].is_successful and state["retries"] >= 2:
        state["retries"] = 0
        return [replan_step]
    return [noop_step]

def pev_loop_done(_outputs) -> bool:
    """所有子任务都通过验证后，退出 Loop。"""
    return len(pev_wf.session_state["plan"]) == 0

pev_wf = Workflow(
    name="pev",
    session_state={"plan": [], "intermediate": [], "retries": 0,
                   "last_q": "", "last_obs": "", "last_verdict": None},
    steps=[
        Step(name="plan", executor=plan_step),
        Loop(
            name="pev_loop",
            steps=[
                Step(name="pev_execute", executor=pev_execute),
                Step(name="pev_verify", executor=pev_verify),
                Router(name="decide_next", selector=pev_router),
            ],
            end_condition=pev_loop_done,
        ),
        Step(name="synthesize", executor=synth_step),
    ],
)
```

拓扑变成：



它真正带来的系统收益

PEV 的本质是：**让错误不再静默传播。** 在普通 Planning 里，一次失败可能污染整个剩余流

程；在 PEV 里，失败在局部被识别，并被重定向回 re-plan 路径。

失败模式

- 额外成本高
- verifier 本身可能误判
- 过度验证会拖慢系统
- 某些任务里"验证"本身比"执行"更难

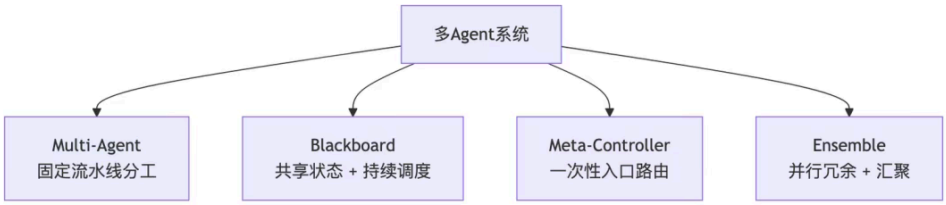
什么时候升级？

如果你接下来的问题不是"执行步骤会不会失败"，而是"一个 agent 根本不该包揽所有认知角色"，那就该进入多 agent 时代。

> 📎 参考实现：**06_PEV.ipynb**

9. Multi-Agent Collaboration：把认知分工写进图里

多 agent 的核心不是"多个模型一起跑"，而是把认知分工、调度策略和冗余机制显式写进图结构，真正要分清三件事：固定分工、动态调度、并行冗余。



它要解决什么问题？

当单个 agent 的 prompt 同时容纳太多角色时，系统就会变形，最典型的问题不是 token 不够，而是**角色冲突**，Multi-Agent 的核心价值是：**把认知分工编码到架构里。**

State 是什么？

在 agno 里它就是 `workflow_session_state` 里按角色划分的几块区域：

```
wf = Workflow(
    name="multi_agent",
    session_state={
        "user_request": None,
        "news": None,         # 由 news_analyst 写入
        "tech": None,         # 由 technical_analyst 写入
        "fin": None,          # 由 financial_analyst 写入
        "final_report": None, # 由 report_writer 写入
    },
```

```
    steps=[...],
)
```

不同角色写入不同字段——state 不再只是"共享上下文"，而开始体现**角色边界**。

关键代码

agno 表达这种固定流水线，有两条路：用 `Workflow` 拼，或者用 `Team(mode="coordinate")` 让一个 leader 固定地调度，这里选最直接的做法——`Workflow` + 多个 `Agent`：

```
news_analyst = Agent(
    name="news_analyst",
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[DuckDuckGoTools()],
    instructions="You are a financial news analyst. Produce a concise markdown section on
)

technical_analyst = Agent(
    name="technical_analyst",
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[YFinanceTools()],
    instructions="You are a technical analyst. Produce a concise markdown section on pric
)

financial_analyst = Agent(
    name="financial_analyst",
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[YFinanceTools(income_statements=True, key_financial_ratios=True)],
    instructions="You are a financial analyst. Produce a concise markdown section on fund
)

report_writer = Agent(
    name="report_writer",
    model=OpenAIChat(id="gpt-5-mini"),
    instructions="Compose a final investment memo from the three sub-reports.",
)

def news_step(si):
    out = news_analyst.run(si.message).content
    si.workflow_session_state["news"] = out
    return out

def tech_step(si):
    out = technical_analyst.run(si.message).content
    si.workflow_session_state["tech"] = out
    return out
```

```
def fin_step(si):
    out = financial_analyst.run(si.message).content
    si.workflow_session_state["fin"] = out
    return out

def write_step(si):
    s = si.workflow_session_state
    return report_writer.run(
        f"News:\n{s['news']}\n\nTechnical:\n{s['tech']}\n\nFinancial:\n{s['fin']}"
    ).content

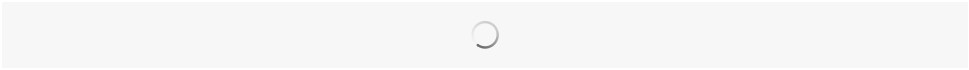
multi_wf = Workflow(
    name="multi_agent",
    session_state={},
    steps=[
        Step(name="news", executor=news_step),
        Step(name="tech", executor=tech_step),
        Step(name="fin", executor=fin_step),
        Step(name="write", executor=write_step),
    ],
)
```

或者，如果你想让"编排也能省掉"，直接用 `Team` 的 coordinate 模式：

```
from agno.team import Team

analysts_team = Team(
    name="analysts",
    mode="coordinate",
    model=OpenAIChat(id="gpt-5-mini"),
    members=[news_analyst, technical_analyst, financial_analyst],
    instructions=[
        "Route each relevant sub-question to the right analyst.",
        "Collect their outputs and synthesize a final investment memo.",
    ],
    markdown=True,
)

analysts_team.print_response("Write an investment memo about AAPL.", stream=True)
```



新增的能力

把一个大 prompt 内部的隐式角色切换，变成多个显式节点。 这带来非常大的工程收益：你可以单独调试某个角色、替换某个角色的 prompt、单独评估某个角色输出、给不同角色不同工具。

失败模式

最大问题是：**流程固定。** 如果执行到一半发现还需要更多新闻背景，固定流水线不会自动返回新闻分析师；如果发现技术分析其实不必要，也不会跳过，解决了"认知拆分"，但没有解决"动态调度"。

什么时候升级？

当多个角色之间的先后顺序本身也需要动态决定时，你需要 Blackboard。

> 📎 参考实现：**05_multi_agent.ipynb**

10. Blackboard：共享状态成为系统中心

它要解决什么问题？

Multi-Agent 里角色有了，但顺序还是硬编码的。Blackboard 解决的是：**不要预先写死专家的调用顺序，而是让共享工作区的当前状态来决定下一步应该激活谁。**

State 是什么？

```
class BlackboardState(BaseModel):
    user_request: str
    blackboard: dict
    next_agent: Optional[str] = None
    is_complete: bool = False
```

之前的 state 更像"按字段分区的结果容器"；现在的 state 变成了一个**共享工作台**。所有专家都围绕这块黑板读写。

关键代码

```
import json

class ControllerDecision(BaseModel):
    next_agent: str = Field(description="One of ['news', 'technical', 'financial', 'write
    reasoning: str = Field(description="A brief reason for choosing the next agent.")

controller = Agent(
    name="controller",
    model=OpenAIChat(id="gpt-5-mini"),
    response_model=ControllerDecision,
```

```
    instructions=(
        "You are the controller of a blackboard system. "
        "Inspect what is currently on the blackboard and decide which specialist "
        "should be called next, or FINISH if the report is ready."
    ),
)

SPECIALISTS = {
    "news": news_analyst,
    "technical": technical_analyst,
    "financial": financial_analyst,
    "writer": report_writer,
}

def controller_step(si):
    s = si.workflow_session_state
    bb_snapshot = json.dumps(s["blackboard"], indent=2, ensure_ascii=False)
    decision = controller.run(
        f"Original request: {s['user_request']}\n\nBlackboard so far:\n{bb_snapshot}"
    ).content
    s["next_agent"] = decision.next_agent
    return decision

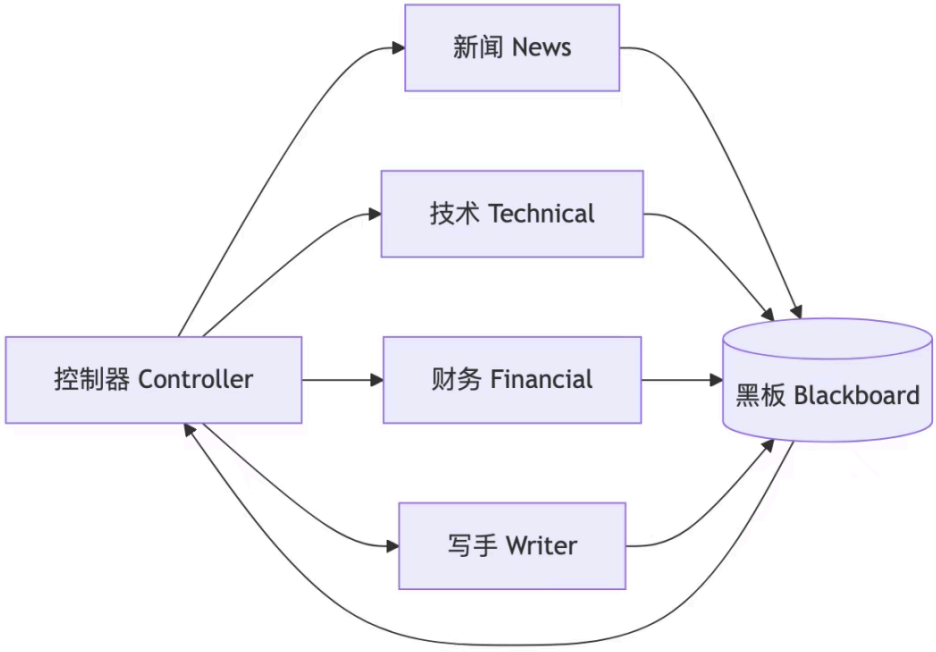
def specialist_step(si):
    s = si.workflow_session_state
    if s["next_agent"] == "FINISH":
        return"finished"
    agent = SPECIALISTS[s["next_agent"]]
    piece = agent.run(
        f"Request: {s['user_request']}\nBlackboard:\n{json.dumps(s['blackboard'])}"
    ).content
    s["blackboard"][s["next_agent"]] = piece
    return piece

def bb_loop_done(_outputs) -> bool:
    """controller 选出 FINISH 就终止循环。"""
    return blackboard_wf.session_state["next_agent"] == "FINISH"

blackboard_wf = Workflow(
    name="blackboard",
    session_state={"user_request": "", "blackboard": {}, "next_agent": None},
    steps=[
        Loop(
            name="bb_loop",
            steps=[
                Step(name="controller", executor=controller_step),
                Step(name="specialist", executor=specialist_step),
            ],
```

```
            end_condition=bb_loop_done,
        ),
    ],
)
```

拓扑不是固定链，而是一个**持续调度回路**：



系统控制中心从 "预定义工作流" 转向 "共享状态 + 调度器"。

新增的能力

持续的、基于中间状态的动态编排。

失败模式

- controller 决策不稳定
- blackboard 结构变脏，信息冲突
- 专家之间重复劳动
- 系统容易过度循环

Blackboard 用灵活性换来了调度复杂度。

什么时候升级/降级？

如果问题的顺序本来就很固定，用 Blackboard 反而是过度设计，但如果你真正需要的不是持续调度，而只是"请求进来时先分诊一次"，那就该看 Meta-Controller。

> 📎 参考实现：**07_blackboard.ipynb**

11. Meta-Controller：一次性路由，而不是持续编排

它要解决什么问题？

很多系统并不需要 Blackboard 那种持续调度，它们的问题更像是：**这条请求是研究类，还是编码类，还是通用问答？**

Meta-Controller 解决的是：**入口分诊。**

State 是什么？

```
class MetaAgentState(BaseModel):
    user_request: str
    selected_agent: Optional[str] = None
    result: Optional[str] = None
```

状态最小而明确：请求是什么、被路由给谁、返回了什么。

关键代码

agno 里这是 `Team(mode="route")` 的原生场景——leader 只做选择，不做任务：

```
from agno.team import Team

generalist = Agent(
    name="generalist",
    model=OpenAIChat(id="gpt-5-mini-mini"),
    role="Handles general Q&A.",
    instructions="Answer general knowledge questions directly.",
)

researcher = Agent(
    name="researcher",
    model=OpenAIChat(id="gpt-5-mini"),
    role="Handles research-heavy queries.",
    tools=[DuckDuckGoTools()],
    instructions="Do multi-step research using search.",
)

coder = Agent(
    name="coder",
    model=OpenAIChat(id="gpt-5-mini"),
    role="Handles Python coding tasks.",
    instructions="Write, explain and debug Python code.",
)

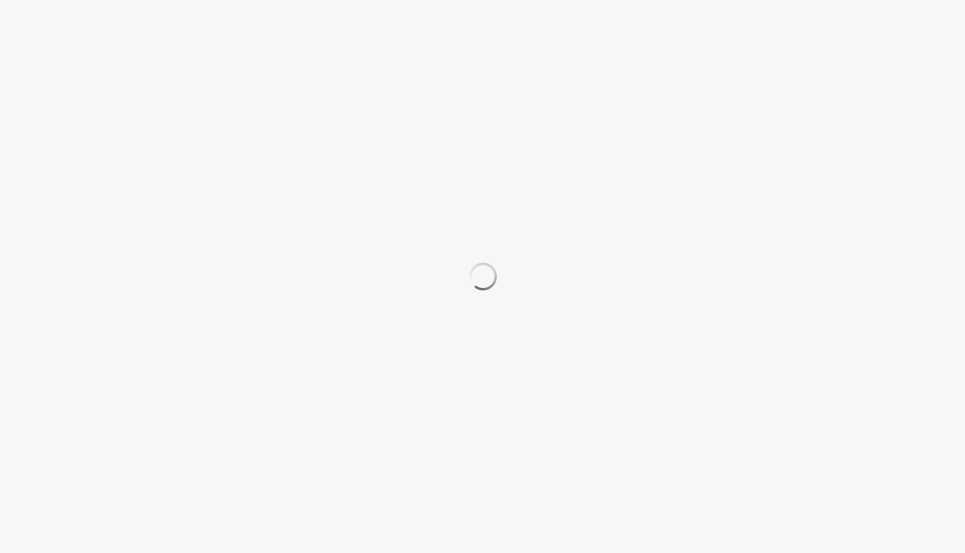
meta = Team(
    name="meta_controller",
    mode="route",                       # 关键：一次性路由
    model=OpenAIChat(id="gpt-5-mini"),
```

```
    members=[generalist, researcher, coder],
    instructions=(
        "Choose exactly one member based on the request type: "
        "generalist for general Q&A, researcher for research-heavy queries, "
        "coder for Python coding tasks. Do not do the work yourself."
    ),
)

meta.print_response("Write a Python function to compute the LCM of two numbers.")
```

`mode="route"` 的精髓不在 graph 多复杂，而在于：**controller 自己不做任务，只做选择。**

拓扑



它和 Blackboard 的本质区别

- `Meta-Controller`：一次性路由
- `Blackboard`：持续调度

前者像"分诊台"，后者像"总控台"。

失败模式

最核心的就是路由错误，因为它只路由一次，所以一旦第一跳错了，整个请求路径就错了，而且这类错误常常不是显式报错，而是"回答得像那么回事，但方向错了"。

它为什么在生产里特别常见？

因为它足够简单，也足够有效，如果让我在生产系统里给多 agent 设计一个起步架构，我通常会先选 Meta-Controller，而不是一上来就上 Blackboard。

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号

> 📎 参考实现：**11_meta_controller.ipynb**

12. Ensemble：它不是分工，而是冗余

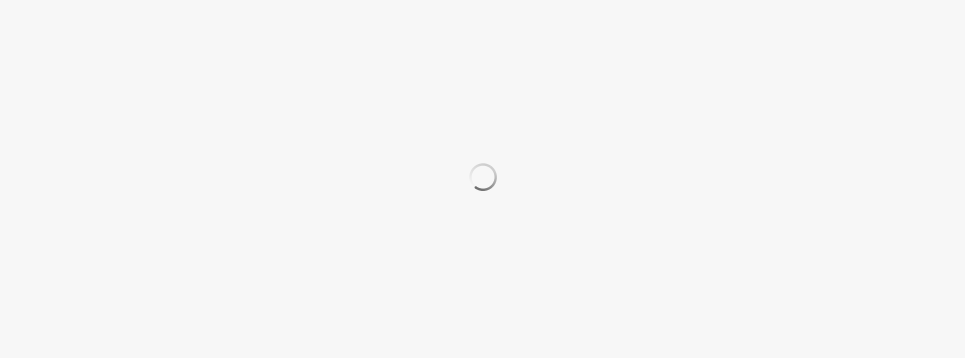
它要解决什么问题？

前面的多 agent 基本都在解决"分工"问题，Ensemble 解决的是另一类问题：**同一个问题，单个 agent 的结论不够可靠。**

你不是担心"一个 agent 做不完"，而是担心"一个 agent 看法太单一、偏差太大、幻觉太重"。

拓扑

标准 fan-out / fan-in：



关键代码

agno 的 `Workflow` 支持 `Parallel` step 做 fan-out，再用一个 aggregator step 做 fan-in：

```
from agno.workflow.v2 import Workflow, Step, Parallel

class FinalRecommendation(BaseModel):
    final_recommendation: str
    confidence_score: float
    synthesis_summary: str
    identified_opportunities: List[str]
    identified_risks: List[str]
```

```
bullish = Agent(name="bullish", model=OpenAIChat(id="gpt-5-mini"),
    instructions="You are a growth-oriented bullish analyst. Argue why to invest.")
value = Agent(name="value", model=OpenAIChat(id="gpt-5-mini"),
    instructions="You are a value analyst. Focus on margin of safety.")
quant = Agent(name="quant", model=OpenAIChat(id="gpt-5-mini"),
    instructions="You are a quantitative analyst. Focus on ratios and trends.")

cio = Agent(name="cio", model=OpenAIChat(id="gpt-5-mini"),
    response_model=FinalRecommendation,
    instructions=("You are the CIO. Synthesize three analyst views into one final "
                  "recommendation. Do NOT hide disagreement - explicitly list risks."))

def run_one(agent):
    def _step(si):
        out = agent.run(si.message).content
        # Parallel 各分支并发写入共享 dict
        si.workflow_session_state.setdefault("views", {})[agent.name] = out
        return out
    return _step

def aggregate_step(si):
    views = si.workflow_session_state.get("views", {})
    body = "\n\n".join(f"[{name}]\n{view}"for name, view in views.items())
    return cio.run(f"Question: {si.message}\nAnalyst views:\n{body}").content

ensemble_wf = Workflow(
    name="ensemble",
    session_state={"views": {}},
    steps=[
        Parallel(
            name="analysts",
            steps=[
                Step(name="bullish", executor=run_one(bullish)),
                Step(name="value", executor=run_one(value)),
                Step(name="quant", executor=run_one(quant)),
            ],
        ),
        Step(name="cio_synth", executor=aggregate_step),
    ],
)

ensemble_wf.run(message="Should I buy AAPL right now?")
```

它和 Multi-Agent 到底差在哪?

- `Multi-Agent`：不同 agent 做不同子任务
- `Ensemble`：不同 agent 分析同一个问题

前者是**分工**，后者是**冗余**。

新增的能力

用多视角和冗余来降低单次推理偏差。 和机器学习里的 ensemble 很像：不是因为单个模型不能输出，而是因为多个模型的误差不完全相关。

失败模式

- 成本线性增长
- 多个 agent 可能共享同样的偏见
- aggregator 可能强行合并不该合并的冲突
- 最终"综合意见"可能掩盖关键分歧

所以 ensemble 的关键不是"取平均"，而是**保留冲突信息并解释冲突**，这就是为什么 `FinalRecommendation` 里必须有 `identified_risks`。

什么时候用?

高风险判断、事实核查、投资建议、复杂研究结论——凡是你不想把决定押在一次输出上的场景。

> 📎 参考实现：**13_ensemble.ipynb**

13. Episodic + Semantic Memory：记忆不是把对话塞回上下文

记忆系统不是"外挂知识库"，而是 state 的外延扩展，真正重要的不是存了多少，而是系统如何区分事件、事实，以及如何避免错误长期固化。

它要解决什么问题?

前面的架构都默认一个前提：当前对话结束后，系统基本失忆，用户要的是：**记得我的偏好、记得我们之前讨论过什么、记得哪些事实在长期上是稳定的。**

这里有两种记忆：

- **Episodic memory**：记住发生过什么（事件摘要 → 向量库）
- **Semantic memory**：记住什么是真的（实体关系 → 图结构/结构化存储）

关键代码

agno 的 `Memory` 负责长期用户偏好/事件，`AgentKnowledge` 负责向量检索，两者结合可以直接落出一套双记忆系统：

```
from agno.agent import Agent
from agno.memory.v2.memory import Memory
```

```python
from agno.memory.v2.db.sqlite import SqliteMemoryDb
from agno.knowledge.text import TextKnowledgeBase
from agno.vectordb.lancedb import LanceDb, SearchType
from agno.embedder.openai import OpenAIEmbedder
from agno.storage.sqlite import SqliteStorage

# Episodic: 用户层长期记忆，自动从对话中抽取
memory = Memory(
    db=SqliteMemoryDb(table_name="user_memories", db_file="tmp/memory.db"),
    model=OpenAIChat(id="gpt-5-mini"),
)

# Semantic: 结构化知识库（这里以文本 + 向量库为例）
knowledge = TextKnowledgeBase(
    path="data/facts",
    vector_db=LanceDb(
        table_name="facts",
        uri="tmp/lancedb",
        search_type=SearchType.hybrid,
        embedder=OpenAIEmbedder(id="text-embedding-3-small"),
    ),
)
knowledge.load(recreate=False)

mem_agent = Agent(
    name="memorized",
    model=OpenAIChat(id="gpt-5-mini"),
    memory=memory,
    enable_agentic_memory=True,          # agent 可以主动写入/检索 episodic
    enable_user_memories=True,
    knowledge=knowledge,                 # semantic
    search_knowledge=True,
    add_history_to_messages=True,
    num_history_responses=5,
    storage=SqliteStorage(table_name="sessions", db_file="tmp/sessions.db"),
    markdown=True,
)

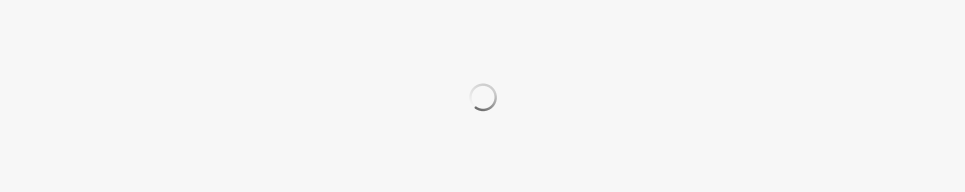
mem_agent.print_response(
    "I'm allergic to peanuts and prefer low-carb meals. Remember that.",
    user_id="alice",
)

# 下一次，即使换了新 session
mem_agent.print_response(
    "Suggest a dinner plan for me.",
    user_id="alice",
)
```

新增的能力

系统的 state 从"图内字段"扩展成：**图内状态 + 图外可检索历史状态。** agent 的能力不再只依赖当前上下文窗口，而依赖它如何从历史里提取相关信息。

工作流



memory 不再是附加模块，而是主控制流的一部分，agno 的 `enable_agentic_memory=True` 就是把这一整条写回链路自动接上。

失败模式

- 错误记忆被写入，造成长期污染
- episodic recall 召回了相似但不相关的信息
- semantic graph 存入过时事实
- 抽取质量差导致记忆结构脏化

memory 让系统更强，但也让错误变得持久。

什么时候还不够?

当你发现向量检索只能"找相似"，却不能做"关系推理"时，就需要 Graph / World-Model Memory。

> 📎 参考实现：**08_episodic_with_semantic.ipynb**

14. Graph / World-Model Memory：当你需要的不是回忆，而是关系推理

它要解决什么问题?

向量检索能回答"哪段历史最像现在的问题"，但不能天然回答"这个实体和那个实体之间隔了几跳关系"。 Graph Memory 解决的是：**把知识从 chunk 组织提升到关系结构。**

技术核心

两步：

1. 从非结构化文本里抽实体和关系
2. 让自然语言问题转成图查询

```
Text -> Knowledge Graph -> Text-to-Cypher -> Query -> Answer
```

关键代码

```
class Node(BaseModel):
    id: str = Field(description="Unique name or identifier for the entity.")
    type: str = Field(description="Entity type, e.g., Person, Company.")

class Relationship(BaseModel):
    source: Node
    target: Node
    type: str = Field(description="Relationship verb in ALL_CAPS, e.g., WORKS_FOR, ACQUIR

class KnowledgeGraph(BaseModel):
    relationships: List[Relationship]

graph_maker = Agent(
    name="graph_maker",
    model=OpenAIChat(id="gpt-5-mini"),
    response_model=KnowledgeGraph,
    instructions=(
        "Extract entities (nodes) and relationships from the given text. "
        "Relationship type should be an ALL_CAPS verb."
    ),
)

# agno 的 Neo4j 工具可以直接做图查询
from agno.tools.neo4j import Neo4jTools

graph_query_agent = Agent(
    name="graph_query",
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[Neo4jTools(url="bolt://localhost:7687", user="neo4j", password="...")],
    instructions=(
        "You answer questions over a Neo4j knowledge graph. "
        "First generate a Cypher query, run it, then synthesize a natural answer. "
        "If the first query returns nothing, rewrite and retry once."
    ),
)

# 1) 抽取
kg = graph_maker.run("Tim Cook is the CEO of Apple. Apple acquired Beats in 2014.").conte
# 2) 写入图库（省略 write_triples），然后查询
graph_query_agent.print_response("Which companies did Apple acquire and in which year?")
```

新增的能力

核心变化是：**从"相似性召回"升级到"结构性推理"。** 这在企业知识问答、组织关系分析、并购链条追踪这类问题上非常关键。

失败模式

- 抽取错误导致图污染
- schema 设计不佳导致图不可用
- Text-to-Cypher 生成错误查询
- 图查询结果正确，但 synthesis 层误读

Graph Memory 的关键不只是 LLM，而是整个知识建模链条。

> 📎 参考实现：**12_graph.ipynb**

15. Tree-of-Thoughts：不是让模型"想更多"，而是让系统"搜更多"

它要解决什么问题？

真正困难的问题不是"链不够长"，而是"路径会分叉，而且需要回溯"，ToT 解决的是：**把推理从单路径生成，升级成对候选路径空间的搜索。**

State 是什么？

```
class ToTState(BaseModel):
    problem: str
    active_paths: List[List["PuzzleState"]]
    solution: Optional[List["PuzzleState"]] = None
```

state 的单位已经不是"一个当前答案"，而是"多条候选路径"。

关键代码

经典的 wolf-goat-cabbage 谜题——标准 BFS 回溯场景，在 agno 里，LLM 负责 *生成候选动作*，程序化代码负责 *搜索树的扩展和剪枝*。 这是 ToT 里重要的架构判断：**不要把搜索控制交给 LLM，交给代码。**

```
import copy

class PuzzleState(BaseModel):
    left_bank: frozenset = Field(default_factory=lambda: frozenset({"wolf", "goat", "cabb
    right_bank: frozenset = Field(default_factory=frozenset)
    boat_location: str = "left"
    move_description: str = "Initial state."

    class Config:
```

```
        arbitrary_types_allowed = True

    def is_valid(self) -> bool:
        dangerous = [("wolf", "goat"), ("goat", "cabbage")]
        unguarded = self.left_bank if self.boat_location == "right"else self.right_bank
        return not any({a, b}.issubset(unguarded) for a, b in dangerous)

    def is_goal(self) -> bool:
        return self.right_bank == frozenset({"wolf", "goat", "cabbage"})

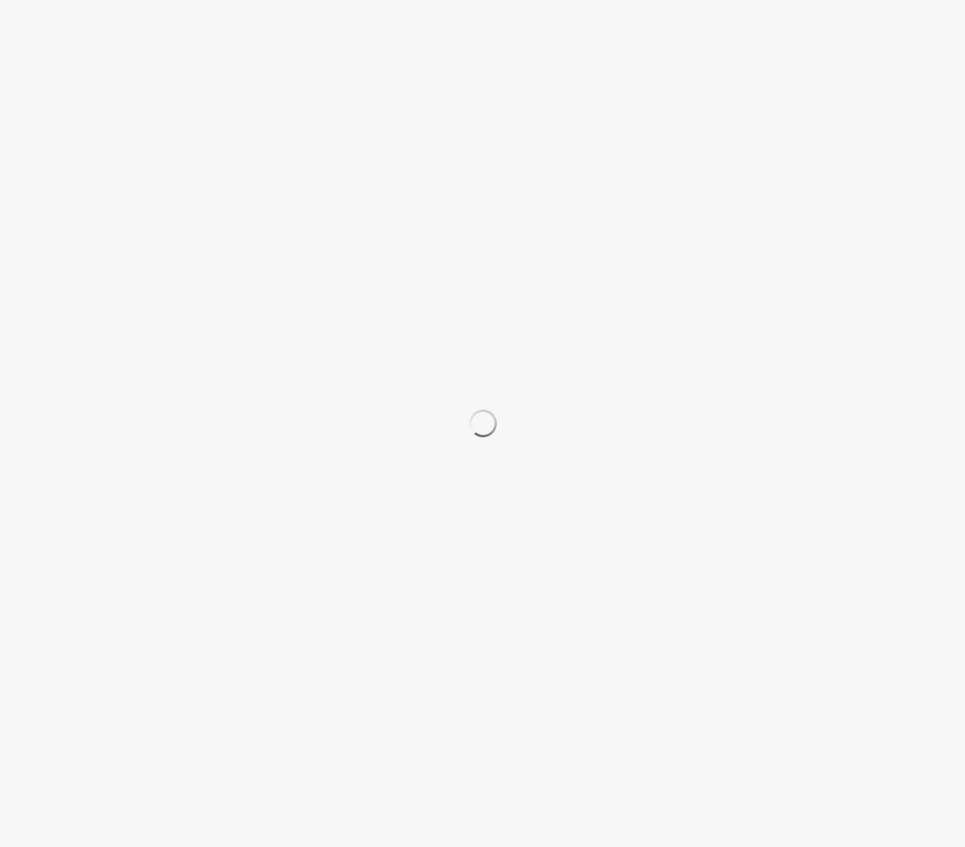
class Proposal(BaseModel):
    moves: List[str] = Field(description="Candidate next moves.")

proposer = Agent(
    name="proposer",
    model=OpenAIChat(id="gpt-5-mini"),
    response_model=Proposal,
    instructions="Given the current state, propose up to 3 distinct next moves.",
)

def expand(state: PuzzleState) -> List[PuzzleState]:
    # 纯程序化展开：LLM 只提供建议，代码保证搜索的正确性
    ...

def tot_solve(initial: PuzzleState, max_depth: int = 10):
    active_paths = [[initial]]
    for _ in range(max_depth):
        new_paths = []
        for path in active_paths:
            for nxt in expand(path[-1]):
                if nxt in path:
                    continue
                new_path = path + [nxt]
                if nxt.is_goal():
                    return new_path
                new_paths.append(new_path)
        active_paths = new_paths
    return None
```

新增的能力



ToT 新增的是：**把推理问题转写成搜索问题。** 这和 ReAct、Planning 都不一样。前者做控制流设计，ToT 做搜索空间设计。

失败模式

组合爆炸，ToT 不是通用架构，只该用于"必须回溯和搜索"的专用场景。

> 📎 参考实现：**09_tree_of_thoughts.ipynb**

16. Mental Loop / Simulator：行动之前，先在内部世界里试错

它要解决什么问题？

在机器人、交易、生产系统配置变更这类任务里，试错有代价。 Mental Loop 要**把试错从真实世界搬进模拟世界**。

关键代码

模拟器就是一个普通 Python 对象，决策 agent 通过工具把它当成"可以快照 可以推进 可以回滚"的沙箱：

```
import copy
import numpy as np

class Portfolio(BaseModel):
    cash: float = 10000.0
```

```
    shares: int = 0
    def value(self, price: float) -> float:
        return self.cash + self.shares * price

class MarketSimulator(BaseModel):
    day: int = 0
    price: float = 100.0
    volatility: float = 0.1
    drift: float = 0.01
    portfolio: Portfolio = Field(default_factory=Portfolio)

    class Config:
        arbitrary_types_allowed = True

    def step(self, action: str, amount: float = 0.0):
        if action == "buy":
            n = int(amount); cost = n * self.price
            if self.portfolio.cash >= cost:
                self.portfolio.shares += n; self.portfolio.cash -= cost
        elif action == "sell":
            n = min(int(amount), self.portfolio.shares)
            self.portfolio.shares -= n; self.portfolio.cash += n * self.price
        self.price *= 1 + float(np.random.normal(self.drift, self.volatility))
        self.day += 1

REAL = MarketSimulator()

def simulate_action(action: str, amount: float, horizon: int = 5) -> str:
    """Roll out an action on a forked copy of the market for `horizon` days."""
    sim = copy.deepcopy(REAL)
    sim.step(action, amount)
    for _ in range(horizon - 1):
        sim.step("hold")
    return f"Simulated value after {horizon} days: ${sim.portfolio.value(sim.price):.2f}"

def execute_action(action: str, amount: float) -> str:
    """Commit the action to the REAL market."""
    REAL.step(action, amount)
    return f"Executed: {action} {amount}. Portfolio now ${REAL.portfolio.value(REAL.price

trader = Agent(
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[simulate_action, execute_action],
    instructions=[
        "Before committing any action with execute_action, first call simulate_action.",
        "If the simulated outcome is worse than holding, do not execute.",
    ],
    show_tool_calls=True,
```

```
)

trader.print_response("Decide whether to buy AAPL today, and how much.")
```

新增的能力

反事实执行（counterfactual execution）。 系统不再直接问"下一步做什么"，而是先问：如果我这么做会发生什么？

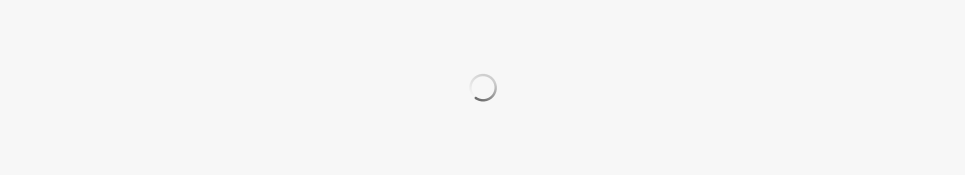
失败模式

simulation-reality gap，模拟器越不真实，agent 越可能做出"模拟里完美，现实里灾难"的决定，这个架构的上限往往不是 LLM，而是 simulator 的保真度。

> 📎 参考实现：**10_mental_loop.ipynb**

17. Dry-Run Harness：真正把副作用关进闸门里

安全架构的核心，不是让 agent 更保守，而是把"是否允许执行"做成显式控制流。 `Dry-Run` 解决外部副作用， `Metacognitive` 解决内部边界感知，它们最好一起出现。



它要解决什么问题？

如果一个 agent 会发邮件、发帖、下单、改配置、删数据，核心问题是：**它在执行前能不能被拦住。**

Dry-Run Harness 把所有真实动作都拆成两种模式：preview 和 execute。

关键代码

agno 里实现 dry-run 有两个关键设计：工具本身带 `dry_run` 参数； `Workflow` 里显式插入一个人工审批 step：

```
import datetime, hashlib

def publish_post(content: str, hashtags: List[str], dry_run: bool = True) -> str:
    """Publish a social media post. If dry_run=True, only preview; no side effects."""
    ts = datetime.datetime.now().isoformat()
    full = f"{content}\n\n" + " ".join(f"#{h}"for h in hashtags)
    if dry_run:
        return f"[DRY RUN @ {ts}] Would publish:\n---\n{full}\n---"
```

```
    post_id = hashlib.md5(full.encode()).hexdigest()[:8]
    return f"[LIVE @ {ts}] Published id={post_id}"

proposer = Agent(
    name="proposer",
    model=OpenAIChat(id="gpt-5-mini"),
    tools=[publish_post],
    instructions=[
        "When asked to post, FIRST call publish_post with dry_run=True to preview.",
        "After the preview, stop and wait for human approval. Do NOT call dry_run=False y
    ],
    show_tool_calls=True,
)

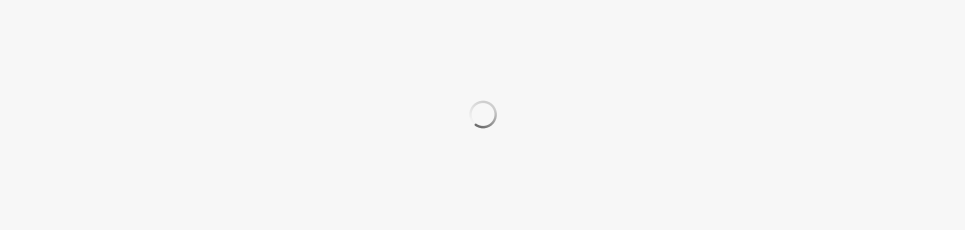
def propose_step(si):
    return proposer.run(si.message).content

def approve_step(si):
    preview = si.previous_step_output
    print(f"\n--- PREVIEW ---\n{preview}\n---")
    decision = input("Approve and go live? (y/n): ").strip().lower()
    si.workflow_session_state["approved"] = (decision == "y")
    return decision

def commit_step(si):
    if not si.workflow_session_state.get("approved"):
        return"Rejected by human reviewer. Nothing was published."
    return proposer.run(
        "Now actually publish the same post by calling publish_post with dry_run=False."
    ).content

dryrun_wf = Workflow(
    name="dryrun",
    steps=[
        Step(name="propose", executor=propose_step),
        Step(name="approve", executor=approve_step),
        Step(name="commit", executor=commit_step),
    ],
)
```

新增的能力



不是更智能，而是：**让副作用在控制流上变成可审批对象。** 很多 agent 系统的问题不在 reasoning，而在 side effects。Dry-Run 把它显式纳入架构。

失败模式

- 人工审批瓶颈
- 预演和真实执行环境不一致
- preview 足够详细反而带来信息泄漏风险

但即便如此，这种架构在真实生产系统里几乎是必须的。

> 📎 参考实现：**14_dry_run.ipynb**

18. Reflexive Metacognitive Agent：系统第一次显式思考自己的边界

它要解决什么问题?

前面的系统都在问"怎么把任务做完"，Metacognitive agent 先问：**这个任务我到底该不该做?** 这是更高级的控制流，因为它不再只对外部世界建模，也开始对自身能力边界建模。

关键代码

```
class MetacognitiveAnalysis(BaseModel):
    confidence: float = Field(description="0.0~1.0, confidence in safely answering.")
    strategy: str = Field(description="'reason_directly' | 'use_tool' | 'escalate'.")
    reasoning: str
    tool_to_use: Optional[str] = None

AGENT_SELF_MODEL = {
    "knowledge_domains": ["general health", "nutrition", "exercise"],
    "tools_available": ["symptom_checker"],
    "confidence_threshold": 0.7,
    "high_risk_topics": ["prescription dosage", "emergency medical advice"],
}

self_model_agent = Agent(
    name="self_model",
    model=OpenAIChat(id="gpt-5-mini"),
    response_model=MetacognitiveAnalysis,
    instructions=(
        f"Your self-model is: {AGENT_SELF_MODEL}. "
        "For each user query, estimate confidence and pick a strategy: "
        "'reason_directly' if confidence >= threshold and topic is low-risk, "
        "'use_tool' if a matching tool exists, 'escalate' otherwise."
    ),
)
```

```
def symptom_checker(symptoms: str) -> str:
    return f"Reference info for: {symptoms}"

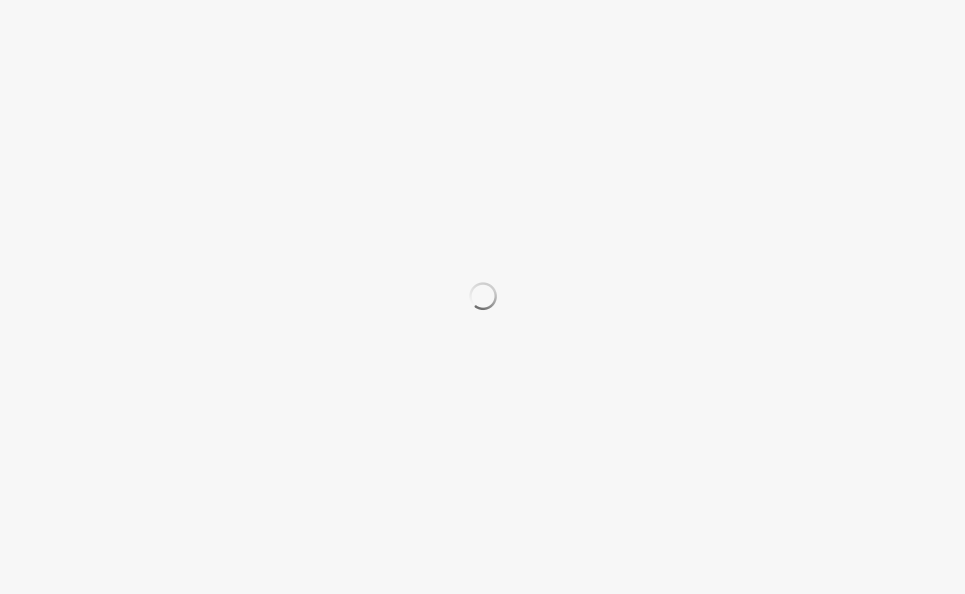
responder = Agent(model=OpenAIChat(id="gpt-5-mini"), tools=[symptom_checker])

def meta_step(si):
    analysis: MetacognitiveAnalysis = self_model_agent.run(si.message).content
    si.workflow_session_state["analysis"] = analysis
    return analysis

def meta_router(si):
    analysis = si.workflow_session_state["analysis"]
    if analysis.strategy == "reason_directly":
        return [Step(name="answer", executor=lambda s: responder.run(s.message).content)]
    if analysis.strategy == "use_tool":
        return [Step(name="tool_answer",
                     executor=lambda s: responder.run(
                         f"Use the {analysis.tool_to_use} tool to help answer: {s.message
                     ).content)]
    return [Step(name="escalate",
                 executor=lambda s: "I'm not confident or allowed to answer this. Escalat

metacog_wf = Workflow(
    name="metacognitive",
    steps=[
        Step(name="self_model", executor=meta_step),
        Router(name="route_strategy", selector=meta_router),
    ],
)
```

新增的能力



它新增的是：**边界感知。** 系统开始知道：我知道什么、不知道什么、需要什么工具、什么时候该把问题交给人类。

失败模式

置信度估计不准，低估自己会过度保守，高估自己会在高风险场景下危险地自信。

它为什么关键？

在医疗、法律、金融这种领域，agent 最强的能力不是"回答"，而是"拒绝"。



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号

> 📎 参考实现：17_reflexive_metacognitive.ipynb

19. Self-Improvement Loop：进化回路

它要解决什么问题？

Reflection 只有一次 critique pass，一个追求高质量输出的系统需要的是：生成 → 评估 → 修订 → 再评估 → 不达标继续，也就是把质量优化做成**进化回路**。

关键代码

最关键的是终止条件，agno 的 `Loop.end_condition` 天然支持这个：

```
class EmailCritique(BaseModel):
    is_approved: bool
    feedback: str

class MarketingEmail(BaseModel):
    subject: str
    body: str

generator = Agent(name="email_gen", model=OpenAIChat(id="gpt-5-mini"),
    response_model=MarketingEmail,
    instructions="Write a marketing email. Follow feedback if provided.")

critic = Agent(name="email_critic", model=OpenAIChat(id="gpt-5-mini"),
    response_model=EmailCritique,
    instructions="Approve only if subject is compelling AND body has a clear CTA AND tone
```

```python
# 跨任务：高质量样例库
class GoldStandardMemory:
    def __init__(self):
        self.examples: List[MarketingEmail] = []

    def few_shot_block(self) -> str:
        if not self.examples:
            return"No gold examples yet."
        return"\n\n---\n\n".join(
            f"Subject: {e.subject}\nBody:\n{e.body}"for e in self.examples[-3:]
        )

    def add(self, e: MarketingEmail):
        self.examples.append(e)

GOLD = GoldStandardMemory()

def gen_step(si):
    state = si.workflow_session_state
    prior = state.get("last_email")
    critique = state.get("last_critique")
    prompt = f"Task: {si.message}\nGold examples:\n{GOLD.few_shot_block()}\n\n"
    if prior and critique:
        prompt += (f"Previous draft:\nSubject: {prior.subject}\nBody:\n{prior.body}\n"
                   f"Critic feedback: {critique.feedback}\nRewrite to address the feedbac
    email: MarketingEmail = generator.run(prompt).content
    state["last_email"] = email
    state["revision"] = state.get("revision", 0) + 1
    return email

def critic_step(si):
    email = si.workflow_session_state["last_email"]
    verdict: EmailCritique = critic.run(
        f"Review:\nSubject: {email.subject}\nBody:\n{email.body}"
    ).content
    si.workflow_session_state["last_critique"] = verdict
    if verdict.is_approved:
        GOLD.add(email)
    return verdict

def should_stop(_outputs) -> bool:
    """Loop 终止：critic 批准，或达到最大修订次数。"""
    state = self_improve_wf.session_state
    last = state.get("last_critique")
    if last is not None and last.is_approved:
        return True
    if state.get("revision", 0) >= 3:
```

```
        return True
    return False

self_improve_wf = Workflow(
    name="self_improve",
    session_state={"revision": 0},
    steps=[
        Loop(
            name="refine_loop",
            steps=[
                Step(name="gen", executor=gen_step),
                Step(name="critic", executor=critic_step),
            ],
            end_condition=should_stop,
        ),
    ],
)
```

新增的能力

两层：**单次任务内迭代优化 + 跨任务积累高质量样例**（`GoldStandardMemory` 把批准过的结果注入未来生成），这是它和 Reflection 的本质区别。

失败模式

- critic 标准不稳
- revision 收益递减
- 记忆库积累低质量样本会反向污染未来生成

Self-Improve 不是"会自动越来越好"，而是"有机会在严格约束下越来越好"。

> 📎 参考实现：**15_RLHF.ipynb**

20. Cellular Automata：LLM 退出主循环，智能从局部规则中涌现

它要解决什么问题？

前面所有架构都默认存在一个中心 agent、一个主控制流、一个 orchestrator。 Cellular Automata 彻底打破这个前提：**有些问题根本不需要中心规划，而适合通过局部规则产生全局行为。**

关键代码

```
class CellAgent(BaseModel):
    type: str  # 'EMPTY' | 'OBSTACLE' | 'GOAL'
    pathfinding_value: float = float("inf")
```

```
    def update(self, neighbors: List["CellAgent"]):
        if self.type == "OBSTACLE":
            return
        if self.type == "GOAL":
            self.pathfinding_value = 0
            return
        m = min((n.pathfinding_value for n in neighbors), default=float("inf"))
        self.pathfinding_value = min(self.pathfinding_value, m + 1)

def neighbors_of(snapshot, r, c):
    """四邻（上下左右），越界跳过。"""
    H, W = len(snapshot), len(snapshot[0])
    out = []
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if0 <= nr < H and 0 <= nc < W:
            out.append(snapshot[nr][nc])
    return out

def run_ca(grid, steps=50):
    for _ in range(steps):
        snapshot = [[copy.deepcopy(c) for c in row] for row in grid]
        for r in range(len(grid)):
            for c in range(len(grid[0])):
                grid[r][c].update(neighbors_of(snapshot, r, c))
```

在 agno 里，LLM 退到了纯"规则设计者/解释者"的位置：

```
rule_designer = Agent(
    name="rule_designer",
    model=OpenAIChat(id="gpt-5-mini"),
    instructions=(
        "You design local update rules for a cellular automaton. "
        "Given a high-level objective, propose a minimal Python update function "
        "that each cell will execute based only on its neighbors."
    ),
)

rule_designer.print_response(
    "Design a CA rule so that every empty cell eventually stores the shortest "
    "path length to the nearest GOAL, treating OBSTACLE as impassable."
)
```

新增的能力

这不是在现有控制流上加一个模块，而是整个范式切换：**从中心控制转向分布式涌现。**

它和前面所有架构最大的区别

LLM 退出了执行主回路，它最多负责规则设计、参数选择、系统解释，真正的求解过程由大量局部节点同步演化完成。

失败模式

- 局部规则设计不当
- 系统收敛慢
- 涌现出错误的全局结构

但一旦规则设计得对，它能解决很多用中央 planner 反而很笨重的问题。

> 📎 参考实现：**16_cellular_automata.ipynb**

21. Evaluator 不是可选项

这个项目反复提醒你：agent 不是只要能跑就行，必须能评估，至少有五类 evaluator：

1. **LLM-as-a-Judge**：用独立 Agent 打分
2. **内置 critic**：直接控制循环是否继续（`Loop.end_condition`）
3. **程序化验证**：像 `is_valid()` / `is_goal()` 这种硬约束
4. **Human-in-the-Loop**：用人工审批做最后闸门（Dry-Run）
5. **演示式验证**：用多场景运行验证系统行为

为什么 evaluator 是核心，不是附属？

因为一旦系统开始反思、重规划、继续迭代、决定是否执行、决定是否升级人类，你就进入"闭环系统"了。 闭环系统如果没有 evaluator，就不知道何时停、何时改、何时拒绝。

没有 evaluator 的 agent，大概率只是一个会循环的 prompt，不是一个可靠的系统。

22. 架构演化表



架构	新增的关键能力	解决的问题	agno 对应能力
Reflection	critique pass	单次生成质量不稳	3 × `Agent` 串联成 `Workflow`
Tool Use	world interface	模型无法触达真实世界	`Agent(tools=[...])`
ReAct	observation loop	工具结果不能驱动下一步	`Agent(tools=..., reasoning=True)`
Planning	explicit plan state	缺少全局步骤控制	`Agent(response_model=Plan)`

架构	新增的关键能力	解决的问题	agno 对应能力
			+ Loop
PEV	verification loop	执行失败会静默传播	Router + verifier Agent
Multi-Agent	role decomposition	单 prompt 角色冲突	多 Agent 或 Team(mode="coordinate")
Blackboard	shared workspace + dynamic controller	固定流水线不够灵活	workflow_session_state + controller Router
Meta-Controller	entry routing	请求类型不同需要分诊	Team(mode="route")
Ensemble	parallel redundancy	单一答案不够可靠	Workflow(Parallel(...)) + aggregator
Episodic/Semantic Memory	long-term recall	系统跨轮失忆	Memory + AgentKnowledge
Graph Memory	relational reasoning	相似召回不能做关系推理	Neo4jTools + Cypher agent
ToT	search tree	线性推理无法回溯	程序化搜索 + proposer Agent
Mental Loop	counterfactual execution	真实试错成本太高	双工具 simulate_action / execute_action
Dry-Run	side-effect gating	副作用动作不能直接执行	工具带 dry_run 参数 + approval Step
Metacognitive	self-boundary reasoning	系统不知道自己不会什么	response_model=MetacognitiveAnalysis + Router
Self-Improvement	iterative quality loop	一次优化不足	Loop(end_condition=...) + gold memory
Cellular Automata	decentralized emergence	中央控制不适合某些问题	LLM 只设计规则，程序化并行更新

23. 怎么选？问你缺哪种控制能力，不要问哪个架构好

你缺的能力	优先架构	为什么
输出质量不稳	Reflection	最小质量闭环
多步工具推理	ReAct	观察-行动循环最实用
全局步骤控制	Planning	把控制流显式化
工具容错	PEV	把验证接进主回路
角色分工	Multi-Agent	把认知拆开

你缺的能力	优先架构	为什么
动态编排	Blackboard	基于共享状态调度
请求分诊	Meta-Controller	一次路由最省复杂度
高可靠结论	Ensemble	用冗余降低偏差
跨轮记忆	Episodic / Semantic Memory	把历史纳入系统
关系推理	Graph Memory	支持多跳查询
回溯搜索	ToT	适合分支型解空间
行动前模拟	Mental Loop	降低真实试错成本
副作用审批	Dry-Run	先预演再执行
边界感知	Metacognitive	先判断能不能做
长期自我改进	Self-Improvement	质量循环 + 样例积累
去中心化求解	Cellular Automata	用局部规则换全局行为

如果你缺的是输出质量

- 先上 `Reflection`；需要多轮逼近和长期改进，上 `Self-Improvement`

如果你缺的是与世界交互

- 简单任务上 `Tool Use`；多步动态任务上 `ReAct`

如果你缺的是显式步骤控制

- 上 `Planning`；工具不可靠，再升级到 `PEV`

如果你缺的是角色分工

- 固定分工：`Multi-Agent`；动态持续调度：`Blackboard`；入口分诊：`Meta-Controller`；同题多视角冗余：`Ensemble`

如果你缺的是长期状态

- 记历史事件：`Episodic Memory`；做关系推理：`Graph / World-Model Memory`

如果你缺的是求解范式

- 需要回溯搜索：`Tree-of-Thoughts`；需要先模拟后执行：`Mental Loop`；需要去中心化求解：`Cellular Automata`

如果你缺的是安全边界

- 需要副作用审批：`Dry-Run Harness`；需要知道自己不能做什么：`Metacognitive Agent`

24. 最终结论：Agent 架构的演化，本质上是在设计更好的控制流

看完整个项目并用 agno 重写一遍后，我最大的感受不是"agent 真多"，而是一个更朴素的判断：

所谓 agent architecture，不是模型能力表，而是控制流设计史。

它在不断回答同一组问题：什么时候该停？什么时候该继续？什么时候该重试？什么时候该换角色？什么时候该查工具？什么时候该调用历史？什么时候该先模拟？什么时候该拒绝？什么时候该让人类接管？

这些问题不依赖某一个框架。无论你底层用什么 DSL，只要开始做真实的 agent 系统，就一定会亲手长出同一组抽象：`Workflow.steps` 表达确定性边 `Router` 表达条件路由 `Loop` 表达循环与终止条件 `Parallel` 表达并行冗余 `workflow_session_state` 表达显式共享状态 `Agent(tools=...)` 内置 tool loop `Team(mode="route")` 把一整段分诊逻辑改为一个 Team 组合

但控制流本身没有变。**这恰恰说明这些问题不是某个框架发明出来的，而是真实系统演化过程中必然会长出来的。**

从这个角度看，这 17 种架构并不神秘。它们只是对同一个系统问题的 17 种不同回答。

如果让我把整篇文章压缩成三句话：

1. **先别迷信"万能 agent"，先把状态和控制流画清楚。**
2. **大多数系统从 ReAct 起步，但可靠系统一定会引入验证、记忆和边界控制。**
3. **真正高级的 agent，不是更敢做事，而是更知道什么时候不该做。**

一旦你看懂这条路径，后面再看到任何新的"agent 架构名词"，你都可以先问它三个问题：

- 它新增了什么 state?
- 它新增了什么 router?
- 它新增了什么 evaluator?

只要这三个问题答不出来，它大概率就不是一种新架构，只是旧架构换了个名字。

参考

1、**https://github.com/FareedKhan-dev/all-agentic-architectures**

2、**https://github.com/agno-agi/agno**



腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号



阅读 1.3万

留言

写留言

还没有留言，来写首评